

CS611 (Software Quality Engineering)

Topic 89-132 quiz 3 fall 2023

Project Planning

Project planning includes a wide range of activities including development efforts, quality assurance and demo dates. Planning for development activities requires estimation containing man-hour (and budgetary) requirement to complete the development. This estimation is prerequisite to the project planning. A point to be noted here is; the estimations are totally different from timelines. 200 hours does not mean 5 weeks. The estimation is usually worked out in terms of man-hours while the timeline is progress of activities on a calendar timescale.

Example of Project Planning

Start Date	1/2/2017
End Date	15/4/2017

No.	Task	Estimate in hours	Start Date	End Date	Assigned To	Status	Comments
1	Add sorting filters all over the listings						
	- Design work	12	1-Feb-17	2-Feb-17		Completed	
2	Deactivated users module						
	- add tab for Deactivated user	4	2-Feb-17	2-Feb-17			
	- list for the deactivated users	8	3-Feb-17	4-Feb-17		Completed	

	- Add time filters (date only)	6	4-Feb-17	5-Feb-17		Completed	- assuming that we don't have any deletion and edit function
3	Analytical report time filters					Completed	
	- add drop down	2	5-Feb-17	6-Feb-17		Completed	
	- handle change listeners on all dropdowns	5	6-Feb-17	6-Feb-17		Completed	
	- Average karma points spent By users	6	7-Feb-17	7-Feb-17		Completed	
	- Total karma points given to projects	3				Completed	
	- Total karma points given to participaid	3				Completed	
	- Total karma points spent on wishes	12				Completed	
	- Total karma points spent on offers	3				Completed	
	- Total karma points earned on sign up	8				Completed	
	- Total karma points earned on needs	24				WIP	
	- Total karma points earned on inviting	3				WIP	
4	Participaid Profile page (CMS)						
	- Participaid link on menu	3				Pending	
	- Design integration / template design	8				Pending	
	- add slider on front end	16				Pending	
	- Admin side option in menu	2				Pending	
	- admin side Form to add image text and link	12				Pending	
	- News widget for frontend	6				WIP	
	- Admin side form to add title detail image and link	48				Pending	
	- show total number of active users of participaid	4				Pending	
	- show total number of active projects of participaid	4				WIP	
5	Hide participaid profile page	5				Pending	
6	Design change and addition	48					
7	QA						
8	Internal Demo						

9	External Demo						
	Total Hours	255					

Project Management and Testing	48
Total Hours	$255 + 48 = 303$
Total Man Days	$303 \div 8 = 37.875$

Example of Project Planning

Start Date	1/2/2017
End Date	15/4/2017

No.	Task	Estimate in hours	Start Date	End Date	Assigned To	Status	Comments
1	Add sorting filters all over the listings						
	- Design work	12	1-Feb-17	2-Feb-17		Completed	
2	Deactivated users module						
	- add tab for Deactivated user	4	2-Feb-17	2-Feb-17			
	- list for the deactivated users	8	3-Feb-17	4-Feb-17		Completed	

	- Add time filters (date only)	6	4-Feb-17	5-Feb-17		Completed	- assuming that we don't have any deletion and edit function
3	Analytical report time filters					Completed	
	- add drop down	2	5-Feb-17	6-Feb-17		Completed	
	- handle change listeners on all dropdowns	5	6-Feb-17	6-Feb-17		Completed	
	- Average karma points spent By users	6	7-Feb-17	7-Feb-17		Completed	
	- Total karma points given to projects	3				Completed	
	- Total karma points given to participaid	3				Completed	
	- Total karma points spent on wishes	12				Completed	
	- Total karma points spent on offers	3				Completed	
	- Total karma points earned on sign up	8				Completed	
	- Total karma points earned on needs	24				WIP	
	- Total karma points earned on inviting	3				WIP	
4	Participaid Profile page (CMS)						
	- Participaid link on menu	3				Pending	
	- Design integration / template design	8				Pending	
	- add slider on front end	16				Pending	
	- Admin side option in menu	2				Pending	
	- admin side Form to add image text and link	12				Pending	
	- News widget for frontend	6				WIP	
	- Admin side form to add title detail image and link	48				Pending	
	- show total number of active users of participaid	4				Pending	
	- show total number of active projects of participaid	4				WIP	
5	Hide participaid profile page	5				Pending	
6	Design change and addition	48					
7	QA						
8	Internal Demo						

9	External Demo					
	Total Hours	255				

Project Management and Testing	48
Total Hours	$255 + 48 = 303$
Total Man Days	$303 \div 8 = 37.875$

Project Tracking and Control

The purpose of tracking the project is to ensure the project is moving ahead as per the plan. Daily standup meetings (SCRUM Meetings) are planned to maintain daily progress and to improve communication among team members. Besides weekly status meeting are key to execute projects as per plan. Again, project planning is prerequisite to project monitoring and weekly status report is published to document the project status and updated all team members.

Example of Project Tracking and Control

Following is a sample weekly status report showing the planned activities along with completion status.

<u>Sample Weekly Status Report</u>						
Project Name		1880				
Planned Start Date (As per contract)		2-Jul				
Planned End Date(As per contract)		2-Oct				
Actual Start Date:		25-Jul				
Estimated End Date (if different from planned end date)		14th November				
Planned % Completion		90%				
Current Project Completion Status (Mention % complete)		4%				

Milestone# / Phase	Planned Start Date	Actual Start Date	Planned End Date	Estimated End Date (if different from planned end date)	% Completion (if Milestone not completed)	Comments
Phase 1	2-Jul	2-Jul	8-Jul	26-Aug	100%	
Phase 2	9-Jul	18-Aug	15-Jul	27-Aug	100%	
Phase 3	16-Jul	27-Aug	25-Jul	19-Sep	80%	

Depending on Nature of Project Multiple Status Reports can be generated.

Fig-1: Weekly Status Report

Audit of Each Phase of PM Process Area

To determine the project health, project managers perform Project Audit which covers all the process areas of project management. **Project Manager is responsible to provide relevant data for Audit.** The purpose is to evaluate the extent to which project management standards are being followed throughout the project. Further, it also assesses the project quality and the reason to the known problems of the whole project and helps in taking corrective measures. Through this report, whole take get a clear summary of individual phases of the project. Here is a summarized example of project audit:

Sample Project Audit Report								
Project Name	A	B	C	D	D	E	F	G
Scope	yes	No	Yes	Yes	Yes	Yes	Yes	No
Budget	yes	No	Yes	Yes	Yes	Yes	Yes	No
Roadmap	yes	No	Yes	Yes	Yes	Yes	No	No
Demo Plan	yes	No	Yes	No	No	No	No	No
Updated Plan	NA	NA	No	NA	NA	Yes	No	No
Project Health	Good	Critically Deficient	Good	Satisfactory	Satisfactory	Good	Weak	Critically Deficient

Response Glossary	
Critically Deficient:	Serious inability to comply with the expectation.
Weak:	Unable to entirely comply with the expectation.
Satisfactory:	Basic objectives met but room for improvement.
Good:	Exactly as per the expectations.
Very Good:	Exceeded the expectation. Role model for other

The actual project audit report is detailed and includes real-time data for all the phases and finally suggests the improvement required.

Earned Value Management (EVM)

EVM is a tool to provide objectives measures of cost and schedule performance of the whole project. The purpose is timely highlighting the cost and schedule issues so that the remedial actions are taken before it's too late to recover. Earned value is expressed as a budgetary value. Of the work done on a project, but still it's a project management tool rather a financial tool.

The results are presented in a graphical format. An example is hereunder:

Three values are being evaluated here:

- Planned Value (PV)

- Actual Value (AV)
- Earned Value (EV)

CPI and SPI

Schedule Performance Index (SPI) is used to determine project schedule and it is expressed as the ratio of earned value and planned value. Following equation is used to do so:

$$= () \div ()$$

The formula concludes as follows:

- If the SPI is greater than 1, it means the project is ahead of schedule.
- If the SPI is less than 1, it means the project is behind schedule.
- If the SPI is equal to one, it means the project is on time

Cost Performance Index (CPI) is used to determine project budget status and it is expressed as the ratio of earned value and actual value. Following equation is used to do so:

$$= () \div ()$$

The formula concludes as follows:

- If the CPI is less than 1, it means the project is over budget.
- If the CPI is greater than 1, it means the project is under budget.
- If the CPI is equal to one, it means the project is as per the planned budget.

Example:-I of CPI and SPI

Consider the following example. As per the result, the project is over budget and behind schedule

Example 1 of CPI and SPI	
Project Audit - 20-April-2017	
Planned Value (PV) - Planned Effort (Contracted Effort)	206
Actual Value (AV) = Actual Effort from Weekly Status Reports	358*
EVM: Completed Tasks Effort as of Project Plan All Task marked completed in Plan after 6-Week	230**
CPI (EVM/AV)	0.642458101
SPI (EVM/PV)	1.116504854
Result	Over Budget --- Behind Schedule
Over Budget	Actual is greater than what is earned
Behind Schedule	Earned is less than Planned

* Sum of Actual Effort Planned to be completed by 20-April-2016

** Sum of all tasks with Status = Completed by 20-April-2016

Example:-I of CPI and SPI

Consider the following example. As per the result, the project is over budget and behind schedule

Example 2 of CPI and SPI	
Project Audit - 20-May-2017	
Planned Value (PV) -Planned Effort(Contracted Effort)	112
Actual Value (AV)= Actual Effort from Weekly Status Reports	216*
EVM: Completed Tasks Effort as of Project Plan All Task marked completed in Plan after 6-Week	40**
CPI (EVM/AV)	0.185185185
SPI (EVM/PV)	0.357142857
Result	Over Budget --- Behind Schedule
Over Budget	Actual is greater than what is earned
Behind Schedule	Earned is less than Planned

* Sum of Actual Effort Planned to be completed by 20-May-2016

** Sum of all tasks with Status = Completed by 20-May-2016

Cavendish Software Chaos Report

According to the Cavendish software chaos report, the success rate of the software and IT project is getting better in the recent years, but still failure rate touches a high percentage and this is a major point of concern for the industry professional. The mentioned report state three major reason behind the failure and requirement engineering is on top with 87% of failure chances if not done accurately. Requirement engineering poses a big challenge for the industry as doing it properly is a strenuous task in today's dynamic and turbulent environment.

Motivation for Software Requirements

As mentioned above, requirement engineering is easier said than done and become harder task when client is not mindful of the requirement and put requirement documentation on low priority. This leads to ambiguity and uncertainty throughout the project and the chances of failure become so high because the development team is not completely sure about what is to be done. In the absence of proper documented requirements, the actual purpose of software development is lost.

Why to Focus on Requirements?

Requirement engineering is the most important area of the entire software life cycle because error produced in this phase cost a lot in later stages and customer will not get the required product in the required time. Again, as mentioned above, faulty or missing requirements are the biggest reason behind the project failure. The following problems can occur during requirement engineering:

- **Missing Requirements:** if not documented properly, project team may skip requirement(s) and it would be nearly impossible to change the software design afterwards. And sometimes, most critical requirements are missing.
- **Client not Involvement:** due to the client's lack of involvement, requirements can be misunderstood or miscommunicated leading to faulty production.
- **Wrong Requirement:** Due to less technical knowledge at the client's end, the requirements are sometimes technically wrong or they are in conflict with some other requirements.
- **Changing Requirements:** If the client is not completely informed of the requirements, he may make changes in later stages of development that might become difficult to incorporate.
- **Out of Scope Requirements:** The requirements stated by the client may go out of scope of the project.

Effort-wise Distribution of SDLC

A **requirement document** is considered to be an overhead in SDLC. It takes only **10%** of the whole project **budget**. Ideally, there should be a 20% budget for requirement gathering and documentation. Because negligence in this phase will negatively affect the entire project in terms of quality, cost/budget, deliverables, meeting customer requirement and revenue.

Requirement Defined

A requirement can be defined as a feature of the system or a description of something the system is capable of doing in order to fulfill the system's purpose. A point worth understanding is that the requirement engineering is not a technical activity, rather it is purely a communication activity. Different stakeholders from all the facets of software development may have different concerns and point of views. But the challenge is to create a win/win situation for all stakeholders. Another impediment is to

understand the requirement from a nontechnical client because they are difficult to handle because of knowledge issues.

Software Requirement vs Requirements

Software requirements are not similar to the requirement in other domains. Software requirements come with a lot of variation and changes (in all stages of SDLC) as compared to other domains where requirements are completely understood and defined in the beginning of the project and they remain same throughout. The software product is not tangible until the first demo and requirements changes after it. For example, in the construction industry the requirement and design are completely understood and they remain fixed throughout the project. But that's not the case with software development where requirements change during all stages of SDLC.

Attributes of Software Requirements

There are multiple attributes of software requirements and they are explained with examples.

Correct

The requirement should capture the client expectations and there is not be any kind of ambiguity in that. All stakeholders must be communicated openly to correctly analyze the expectations/needs of client and converting them into software requirement. As a good practice, the requirement should be stated in single line and must not exceed 30 to 50 words in length. Open ended and subjective terms should be avoided as they may cause misunderstandings. Terms like 'etc.' or 'assumed' are totally not appropriate for documenting requirements.

Example of Correctness

Requirement: GUI should be user friendly	
Issue	Suggestion
What is user friendliness? Is use of more images or text is desirable? or is there any preferable color scheme?	This sort of requirements are very difficult to be correct in the first cut. Ask client for reference or show him some reference material or sample website to get started with. Avoid falling in the trap of using subjective terms.

Requirements must not come in conflict either with other requirements or the project scope, this characteristic is called being COHERENT

Coherent

Requirements must be coherent, consistent and they must not come in conflict either with other requirements or the project scope. While documenting requirements, vocabulary or any technical terms mentioned in the statement must be consistent and defined so that all stakeholders are on the same page. The requirement should be logical and it should add value in the project

Example of Coherent

Requirement 1	Requirement 2
Customer support should be IT ISO-ITIL 3.0	Customer support should be IT ISO-ITIL 3.0
Question: Which one is correct version of ISO or both are differently correct?	
Answer: Only one standard should be used.	

Completeness

For software requirement engineering, an assumption is the mother of all failures. The requirement should be complete so all stakeholders completely understand them it must not leave any room for guessing and assuming things. To avoid ambiguity, a requirement must express the entire need and state all conditions and constraints under which it applies. Incomplete requirements can lead to faulty system development and ultimately the customer will reject it and users will be unsatisfied.

Example of Completeness

<i>Requirement: The product shall provide status messages at regular intervals not less than every 60 seconds.</i>	
Issue	Suggestion
Requirement is incomplete Open Question: What are status messages and where are they supposed to be displayed?	Status messages to be displayed on designated areas on UI or they should come as Pop-up messages. <i>Requirements should be closed ended as much as possible</i>

Feasible

A requirement is considered to be feasible if it is possible to implement it within the scope and limitation of the project. Or in other words, a requirement would be feasible if it can be satisfied. The best practice is to involve and engage developer(s) to provide technical insights about the requirement. It has been said that having zero bug in a software product is not a feasible requirement as it may take a long time to prove it.

Example of Feasible

<i>Requirement: The product shall switch between displaying and hiding non-printing characters simultaneously</i>	
Issue	Suggestion
<u>Not Feasible</u> Reason: Computers cannot do anything simultaneously, so this requirement is not feasible	The user shall be able to toggle between displaying and hiding all HTML tags and non-HTML tags <u>It is very difficult to identify feasible or non-feasible requirement without technical input.</u>

Necessary

A requirement must be necessary for the software system and should be adding some business value. This type of requirement comes from the business goals and the need behind developing the software solution. In the absence of such requirement, the system would not be able to function properly as per the need of the client. So the project team must work on tasks that fulfill client needs instead of working on what they want.

Example of Necessary

<i>Requirement: Clients want login integration via social media (any one) beside normal sign-in</i>	
Issue	Suggestion
Developer become excited when implementing login via social media and implemented 3 different login via 3-different social media applications and forget normal sign-in. Client totally reject the demo as for Client normal sign-in have more Business value from customer view point	Always follow customer requirements or whatever fulfills the customer's needs. <u>Focus should be on what is expected by client from market view-point.</u>

Verifiable

A requirement can be considered as **verifiable** if it can be implemented in the system and so it is **demo-able**. The quality assurance team should be able to close the requirement after verification. Requirements which are not testable are not verifiable requirements. As a good practice, each requirement must be expressed unambiguously to make it a verifiable requirement and words like ‘must’, ‘shall’ or ‘etc.’ must be avoided while documenting the requirements.

Example of Verifiable

<i>Requirement: Loading time of the website should be as minimum as possible?</i>	
Issue	Suggestion
What is minimum? Minimum is not defined and hence QA can't test it i-e what is value for which QA should test, result there will always be conflict between development and QA team.	Always ask for numeric value, Minimum loading time should be at most <u>200 milliseconds</u> <i>Always look for keywords to be avoided and get requirements vet from QA department also.</i>

Traceable

Being traceable, requirements should be linked to source in Requirement Specifications. A requirement will be considered as unique if it has a unique identifier like some ID number. The traceability feature of requirement enables to ensure the product is built as per the needs/requirements. As linked with the source, the requirements must be linked with Use cases, design and test cases. Requirements should have forward and backward integration. Forward integration is used to identify where does this requirement get used while backward integration is used to know where did this feature originated.

Example of Traceable

Consider the following scenario: After Project fails to deliver on time, process of root-cause analysis (RCA) was initiated to identify reasons due to which team is unable to deliver on time. Core reason which comes out of RCA that audit team is unable to identify the requirements on which key developer was working and developer spend major part of his time on this non-traceable requirement. In Specs or SRS there was no discussion on a feature of which time was spent.

Non-traceable requirements often lead to Project failure – This RCA is from real-time analysis, intentionally names of project is omitted.

Functional Requirements

Functional requirements define what system should do and how the system should behave under certain situation or in a specific environment. In other words, the functional requirements describe the core functionality of the application. Functional requirements drive the application architecture of a system may include calculations, technical details, and other specific functionality that define what a system is supposed to accomplish. **Functional requirements are captured using Use Cases** (a use case is a list of actions or event steps, typically defining the interactions between systems, to achieve a goal.).

Template for Functional Requirements

Following template can be used to document functional requirements:

<u>Sample Template for Capturing Functional Requirement</u>	
Use Case Name:*	
Use Case ID	
Actor:	
Summary:	
Pre-Condition:	
Post-Condition:	
Extend:	
Uses:	
Normal Course of Events:	
Alternative Path:	
Exception:	
<i>* Use cases serve as basis as connector between different phases of SDLC</i>	

Example-I: Filled Functional Requirements

Functional Requirements define particular _____ of the system when conditions are true.

✓ behavior

graph

statics

transaction

Use Case Name:	Sign In
Use Case ID*	1
Actor:	Administrator, Super User, Doctor, Receptionist. Accountant, Lab Manager
Summary:	This use case describes the scenario in which actor logs into the system prior to its usage.
Pre-Condition:	Actor must be already registered and authorized with the system.
Post-Condition:	If the actor enters valid username and password and also already registered and activated user, then he/she will be logged into the system successfully. If not the system state remains unchanged.
Extend:	N/A
Uses:	N/A
Normal Course of Events:	Actor will enter his/her username and password provided by Administrator into the Sign-in form and press Login button. System validates the username and password and redirects the actor to his/her respective home page.
Alternative Path:	If an actor isn't activated currently within the system by Administrator, then system won't let the corresponding actor to Login. Rather system will displays a message to that user "You are not activated anymore"
Exception:	If an actor enters invalid username or password, then system will displays an error.

** Use Case ID is used to make the requirements traceable*

Example-II: Filled Functional Requirements

Use Case Templates are used to generate use case diagrams:

Use Case Name:	Register new patient
Use Case ID	7
Actor:	Receptionist, Administrator
Summary:	This use case describes the scenario in which actor registers a new patient within the system.
Pre-Condition:	Actor must be login prior to perform registration.
Post-Condition:	Patient will be successfully registered in the System if not already registered and will become eligible for Appointment reservation, Doctor Checkup, Lab tests and other applicable processes.
Extend:	N/A
Uses:	Patient registered, Cancel registration
Normal Course of Events:	Actor will enter all the necessary details in the patient registration form and press the "Register" button. System will save this new patient in the system and display a message "Patient Registered Successfully!"
Alternative Path:	If actor doesn't want to register the patient, he will press the "Cancel Registration" button and system redirects the actor to its home page.
Exception:	If the intended patient already registered within the system then system will prompt a message "Patient Already Registered!" and system won't allow registering it again.

Changing Requirements

The release of first demo-able version of the software product leads to Change Request or Bugs from client. The changing or new requirements at this stage are managed using Change Request Template. To fulfill the new requirements, the project team always has to put some extra efforts and before incorporating any new requirement, impact analysis must be performed and the results should be communicated to the clients in order to avoid any conflict in terms of schedule, budget and/or requirement. Bugs should be fixed because they are in scope errors which are committed to be delivered in correct form.

What is software release?

A release can be defined as a distribution of the final version of the software product. The whole software product is divided into multiple releases which include a different set of features environment variables and backlog. Features are also further divided into doable releases. In simple words, Product development is achieved via Release when a working product is delivered to the customer.

How release is build

Each release is considered to be a project with tangible deadlines to be followed. Features are provided by the sales or business development team while environment variables are provided by the technical team. Environment Variable majorly includes next software version to be compatible with the product. Backlogs are discussed and finalized between Sales and Technical Team and communicated the decision to the technical team so it can plan the release.

Release Management

Release management includes the planning, scheduling and execution of the agreed features of the release. The deadlines and timelines are shared with the clients as well. The most important part is the approvals that must be sought on schedule and features to be added in the release. Release management is another name of controlling a software build throughout different stages.

What are release notes in general?

A release note is a document distributed and released as a part of the final product. It contains information about the scope of the software product, performance benchmark, features added/delivered and the known issues that customer might face. The release notes are written by the technical writer, but the QA team is the owner of the document. This document is only distributed once the product or service is carefully tested and approved against the requirement or specification provided by the development team.

What is Software Quality Assurance?

Software Quality Assurance (SQA) is the process to make sure that the developed product is meeting the requirements of the customer and all the features are working as per planned. SQA team is considered to be the internal customer of the product and the product is released only if approved by Quality Assurance (QA) team. Software quality assurance is an ongoing process and it's a part of every stage of Software Development Life Cycle (SDLC) until the product is complete just to make sure that the required quality level is achieved throughout the development process.

What is Software Requirements vs Quality Assurance?

Requirements must be reviewed by the Quality Assurance (QA) resources before finalizing them. QA will review all the requirements and write test cases against all the use cases. As per the modern best practices, QA must be involved since the initial stages of the project and QA must be aware of what is being built and how it is being built so they can prepare the quality assurance activities in a planned and organized way. So it won't be wrong in saying that QA is a major stakeholder in the gathering and finalizing software requirements.

Verifying Requirements

Requirement verification is a structured and organized activity in order to confirm that the built software product fully address the documented and agreed upon requirements. The Quality assurance team performs various tests during all the stages of SDLC to ensure that the final product is as per the needs and requirements. To prevent rework, requirements should be validated and approved by QA and all stakeholders before development. That's why software quality assurance is considered to be an ongoing process.

Requirement verifications are formally and structurally achieved through _____

requirement gathering

sales team

development team

 Quality Assurance

Quality Assurance Basics

Software products are not built overnight. It takes a lot of efforts, team coordination, development and testing come up with the final version of the product. Regardless of the software product, its complexity or size, the purpose of Quality Assurance (QA) remains the same; to reduce the defects to minimum to ensure minimum disruption. High priority or the sensitive areas of the product should be defects free, but as mentioned in the last module, it's not possible to have a software product with zero bugs. Through following standards and best practices the number of defects can be reduced to a minimum level but cannot be totally removed from the product.

Quality Assurance Basics

Software products are not built overnight. It takes a lot of efforts, team coordination, development and testing come up with the final version of the product. Regardless of the software product, its complexity or size, the purpose of Quality Assurance (QA) remains the same; to reduce the defects to minimum to ensure minimum disruption. High priority or the sensitive areas of the product should be defects free, but as mentioned in the last module, it's not possible to have a software product with zero bugs. Through following standards and best practices the number of defects can be reduced to a minimum level but cannot be totally removed from the product.

QA and Defect

QA and defects are always tied to each other and their relationship can never vanish. QA always tries to keep defects at a minimum level and for this adopts different approaches to avoid damages to the software product because of defects. The last resort is that the critical functionality of the software product should work at least and defects must not be producing any kind of threat or damage to the critical function of the product.

QA and Defect: Classification Scheme

Defects can be classified into three categories. These are as follows along with a brief explanation of them.

- Defect Prevention - Error Source Removal: **Defect Prevention is the process of addressing root causes of defects to prevent their future occurrence.**
- Defect Detection: it's a process to detect and remove defects as early/many as possible through various approaches like QA, code reviews, code inspection and design review.
- Defect Containment: it measures the amount of defects that any QA team was able to find as a portion of total defects, i.e. found by QA, appeared as run-time error. The focus of this activity is to reduce/eliminating escaping defects.

Pre-release Defects

Pre-release defects are defined as 'Dormant Defects' which have the potential to create problems for users and customers. These inactive defects are triggered in a specific situation or may occur in a low priority area where the concentration of Quality Assurance (QA) activities are low and they might not get specific attention until they are triggered.

Example of Pre-release Defects

Imagine the following scenario: 'The cash register scans thousands of items such as bread, milk and cheese every day without trouble. However, when someone buys products and use the cash + credit card and gift voucher to pay for it? Since this is not a common way of payment it was not tested properly and it had a bug, now whole system is in wait state due to this?'

Pre-release defects if generated at client sites then depending on the impact it has to be fixed, but they are usually part of the release notes.

Release includes _ environment variables and backing.

Post-release Defects

Set of feature SRS DOCUMENT
Release notes DFD diagram

Post-release defects are those which appear in production or at customer site. The software supplier company has to pay from its pocket to fix those post-release defects. Post-release defects have higher cost of fixing compared to other defects because of multiple installation of the product and the defect has to be fixed at every place. Besides financial loss, company image, trust of the company, reputation, market position all of them will be negatively impacted.

Example of Post-release Defects

Consider the following scenario:

'There was a functional requirement to export logs generated from the database to Comma Separated Values (CSV) file so user can analyze it later on. During testing CSV was tested and worked fine but at Production site when generated some CSV records went missing in the files which required hot-fix at the production site.

After careful analysis it was revealed that over 500000 records in CSV more records went missing when writing it from the Database and it was decided to generate separate CSV after 500000 records and in the end merge two files to generate third CSV. During Quality Assurance (QA) CSV was tested for 30000 records because there was no upper limit mentioned in the Software Requirements Specification (SRS).'

Production level bugs have very high cost plus it can hit reputation also.

Defect Prevention Basics

Defect prevention is a way to reduce the number of defects and the cost to fix them. Besides, it also dig deep into the sources of the error and suggest action to avoid them. Defect prevention is based on the

assumption that there are known error sources or inappropriate actions causing defects and error in the software product. The best way to prevent the defect is to detect them as early as possible, find its source and take corrective actions in order to avoid them in future as well.

Defect Prevention: Education and Training

People factor is one of the critical factors in designing, producing and delivering quality software products. This puts a lot of emphasis on team training and education because sometimes lack of skills or relevant training leads to disastrous defects that could be prevented otherwise. Change is constant and it is fast paced to handle changing requirements, needs, and technologies, the software engineers must be aware of the latest trends.

Domain Knowledge

Software developers are not considered to be the domain specialists rather they are pure technical resources. For some projects, domain specialists are hired to work with business analysts and/or developers to give some technical insights related to the domain. Domain specific knowledge is of utmost importance for understanding the requirements as all domains have their own terminologies and their meanings vary from domain to domain. That's why low domain specific knowledge may lead to ambiguity, confusion, and demotivation among the team.

Example of Domain Knowledge

The following scenario will help in understanding the concept:

‘In Property tax calculation Project resident of the property need to provide total covered area, total constructed area, total floors, type of property so that accordingly category and off-road / on-road rates can be applied. Based on these provided data appropriate Law will be applied to calculate Property Tax.’

To derive appropriate rates and formula there is need of domain specialist.

Lack of Expertise in Phases of Software Development Life Cycle (SDLC)

To ensure delivery of Quality Software Product, adherence to best practices of Software Development Life Cycle (SDLC) is the key. Lack of execution details of Phases of the SDLC is the main cause of defects. Lack of knowledge in another reason behind defects as it hinders the learning and adoption of the best practices. For instance Lack of knowledge of Requirement Management or Product integration is the root cause of defects. Besides, there are different variation of SDLC and a person may not have sufficient knowledge of the SDLC methodology and it may affect team performance.

Lack of Process Knowledge

Just like lack of expertise, lack of process knowledge may give the same results. Process knowledge at team and individual level is essential to prevent defects. Any successful project

requires technical skills, management skills, well-defined product requirements and well-defined processes that support project performance, control and improvements. Creation of the process and on-boarding/implementation of the process are two different things. The success of the process is based on how well it is defined and implemented as well. So the process knowledge is important and its absence may lead to errors and defects.

Example Lack of Process Knowledge

Consider the following scenario:

‘During the execution of the Project and after presenting the Demo to the client if Project Manager is not fully-aware about managing the Change Request then it might end-up in asking the developer to write code which is out of scope and in this process in-scope features might get delayed or impacted, resultantly there will be defects in features which were in-scope.’

Awareness about implementation details of the process is very critical for defect prevention.

Defect Reduction

As mentioned in the previous chapters, defect prevention helps in reducing the number of defects, but it doesn't guarantee 100% defect free product. Realistically speaking, there will be defects in the product no matter how much good defect prevention mechanism is adopted. But still it is very important to have a vigorous defect reduction process in-place to remove defects. The aim is to reduce defects as much possible from their sources to prevent them in future.

Inspection: Direct Fault Detection and Removal

Inspection is one of the most commonly used Quality Assurance (QA) activities and it is also a very effective QA alternative to reduce bugs. Inspections are generally conducted to identify smells at any phase of Software Development Life Cycle (SDLC) and usually these are walkthrough of different artifacts to provide **another EYE**. Inspections are applied to code level as well to check their compliance with different artifacts.

“Software inspection deals with software defects already injected into the software system by detecting their presence through critical examination by human inspectors. As a result of this direct examination, the detected software defects are typically precisely located, and therefore can be fixed easily in the follow-up activities.”

Example of Inspection: Direct Fault Detection and Removal

The following scenario will help in understanding the concept:

‘During the development phase of the Project and before Quality Assurance (QA) Process to start, in code review it was revealed that when a login is performed via social media then normal login process generates an error. Multiple developers are working shared code-base and different developers are working on login module. After multiple inspections it was revealed that Plugin

which was used for log-in via social media is changing the status of a variable at Database level, which was used by a team which was working normal login due to which functionality was broken of normal login. After discussions it was decided to use a different plugin for login via social media.'

Decision of using plugins or off-the self-solutions should be made after careful review.

Testing: Fault Detection

Test is defined as a core Quality Assurance (QA) activity with the purpose to identify the faulty areas which are to be fixed. Testing is a process where QA team executes the product to find out defects or bugs and to ensure that the product gives the required output. Testing is conducted at different levels of Software Development Life Cycle (SDLC) but the purpose remains same. Formal testing of the products starts when a development team hands over the code to QA team who perform different testing and sub-testing activities to ensure the stability of the product.

Defect Containment

Despite of vigorous defect prevention and reduction strategies and activities, existence of defects is still possible and it becomes more risky in high impact applications in which the stakes of failure are very high. For example, in different software product for medical, transportation or nuclear industry, the risks associated with defects are very high. Due to modern fault detection and prevention tools and frameworks, few faults are generated under rare conditions in real-time. But still it is impossible to test exhaustively all the conditions and the final possible strategy is to break fault-failure relation to contain the damage.

Software Fault Tolerance

Traditional Hardware systems require a high level of reliability and availability. In case of such system backups are used for reduced functionality in case of system failure. N-Version Programming (NVP) is used to run multiple versions of the same specifications in parallel. If there is a fault in one version then dynamically the faultless version is switched. The N-Version ensures that fault in limited versions does not compromise the global versions. Faults are not identified, hence not fixed which is contradictory to defect prevention.

Safety Assurance and Fault Containment

For safety critical systems the faults in even low-priority areas with consequences has to be avoided. Hazard reduction techniques include automatic locking of some portion of the software. Damage control through escape routes is achieved to ensure minimal global damage.

Defect Resolution

Defect Resolution is a process that keeps record of reported defects till the testing department close it. The traditional methodology typically starts with defect identification with Quality Assurance

team during testing/QA activities, the defect is then sent to the development team to fix, and then again returned to QA to be validated. But sometimes, defects are not solved due to many reasons like lack of communication, misunderstood requirements or inability to remove defects and such cases defect are not fixed and all the parties agree to it.

Defect Lifecycle

A defect lifecycle consists of all the necessary steps to be carried out from defect identification or reporting to defect resolution. All the reported defects are to be fixed, re-verified and formally closed by the QA team. All the defects are tracked by assigning different statuses to them. Like as mentioned above, sometime it is not possible to remove defects, so status like Assigned, In-Progress, Fixed, Not a Fix, Pending, Not Reproducible and Closed are usually used to track defects.

Stakeholders in Defect Lifecycle

The most important factor in defect resolution is that the decision regarding any defect must be accepted by all stakeholders. For example, if it's decided that certain defect is not to be fixed in current release, then all stakeholders must agree to that. Usually minor defects with very less impact are deferred to the next release with mutual consent and even in some cases the reported one also but can be reclassified as 'not a bug' if the situation demands so. The key point is that all the decision regarding defects must be made by mutual consent of all stakeholders.

Defect Logging

This is the very first step in defect resolution and in this step Quality Assurance (QA) team tests the application or the product to identify, discover and report all the defects. A defect log is established containing all the defects identified in the testing phase with necessary parameters like defect ID, priority and severity level, current defect status and others. Defects log also ensures that all the necessary details about defects are provided to the development team so they are able to understand and fix the defect.

Defect Tracking

Defect tracking is a methodology used to track the discovered and reported defects and it ensures that the defects are being monitored and controlled throughout different stages of defect resolution.

This is done by using templates or tools to ensure that tracking is being done properly. Today's modern software products are complex and may contain huge numbers of defects with high level of severity and criticality and this makes defect logging and tracking very important part of software development.

Example of Defect Logging

The Following is a sample template that can be used to log and track defects. This file is usually shared with all team members to improve collaboratio

Sample Defect Tracking & Log Sheet/Template

Defect No.	Date Created	Created By	Defect Description	Steps to Reproduce	PM Process	Lifecycle Phase	Priority	Owner	Assigned Date	Estimated Time to Fix	Status
1	10/19/16	Kamran	Login not working	Click on Login Button-> Valid credentials -> Error Message	Executing/Controlling	Testing	High	Jim	10/19/16		In Progress
2	10/19/16	Saad	Logo is broken	Logo is shown distorted when loaded	Executing/Controlling	Testing	High	Jim	10/19/16		In Progress
3											

Following is a sample template that can be used to log and track defects. This file is shared with all team members to improve collaboration. Usually defects are completed with consensus if there is a conflict.

Sample Defect Tracking & Log Sheet/Template

Defect No.	Date Created	Created By	Defect Description	Steps to Reproduce	PM Process	Lifecycle Phase	Priority	Owner	Assigned Date	Estimated Time to Fix	Status
1	10/19/16	Kamran	Login not working	Click on Login Button-> Valid credentials -> Error Message	Executing/Controlling	Testing	High	Jim	10/19/16		In Progress
2	10/19/16	Saad	Logo is broken	Logo is shown distorted when loaded	Executing/Controlling	Testing	High	Jim	10/19/16		In Progress
3											

Product Based Defects

Defect resolution in a product based scenario is an ongoing activity and remains continued even after the product is deployed on the client's site. A special team is available to provide support after deployment to ensure hot-fixes for defects removal and the support and quality maintenance plan are established in earlier stages of Software Development Life Cycle (SDLC). Post-deployment defects are more critical in nature and cost many times more as compared to the ones rectified in earlier stages of development.

Project Based Defects

In a project based scenario, bugs and defects are identified, reported and fixed during the project lifecycle. In other words, all defects are to be discovered and fixed before handing over the product to the customer. The defects categorized as 'NotAFix' are to be agreed by all stakeholders of the

project. Service level agreements (SLA) are signed by both parties in order to provide post-deployment support.

Product Evaluation

As the name suggests, product evaluation is a process to evaluate the reliability, stability and the market value of the product. It can be further defined as the assessment of the final version of the software product according to the specified procedures (requirement, best practices, software development principals and alike). The key components of product evaluation include different criteria like code quality, sustainability of the product, defect backlog, features, reliability of features, compatibility, support of the features and many others.

Software Maintenance and Defect Handling

Project and product based defect handling is different from each other. Same is the case with support to be provided after deployment of the product. Software maintenance defines the scope of the support to be provided with all relevant terms and conditions.

There are following levels of software support:

Level 1	Customer Support Team: A special team other than QA and development resources with the expertise to provide technical support to customers
Level 2	QA Team: Members from QA team are involved in providing support to customers
Level 3	Development Team: Members from development team are involved in providing support to customers

Tool and Processes in Defect Handling

Usually MS Excel sheets are used to log and track defects, but there are many other free and paid tools available in the market for the same purpose. These tools provide that provide some additional features to make tracking and logging easier. In those tools, support team has the admin rights, and they add QA and Development resources as per the need of the situation.

Using Mantis for Defect Handling

Mantis is one such tool used for keeping logs and tacking details of the defects. This tool is also available on the cloud and it makes collaboration among team members and other stakeholders easier and faster.

A [link](#) has been provided for further information using mantis for defect handling.

Configuring Mantis

Steps for Configuring Mantis:

Click on the given link: <https://www.mantisbt.org/index.php>

Go to the demo tab and click the “Start your free trial” tab.

Sign up/Register with your credentials as shown in the video.

Example-I: Logging Defect in Mantis

Link to the login page: https://www.testqse.mantishub.io/login_page.php

Create User/Login via User:



 Login

Login

Create Project:

We have already pre-created your first project and called it 'MyProject'. However, if you would like to create more projects, you can create them as follows:

- Click on 'Manage' on the sidebar menu
- Click on 'Manage Projects' at the top menu bar
- Click on 'Create New Project' and enter the project name

Figure 1-Interface for "My View"

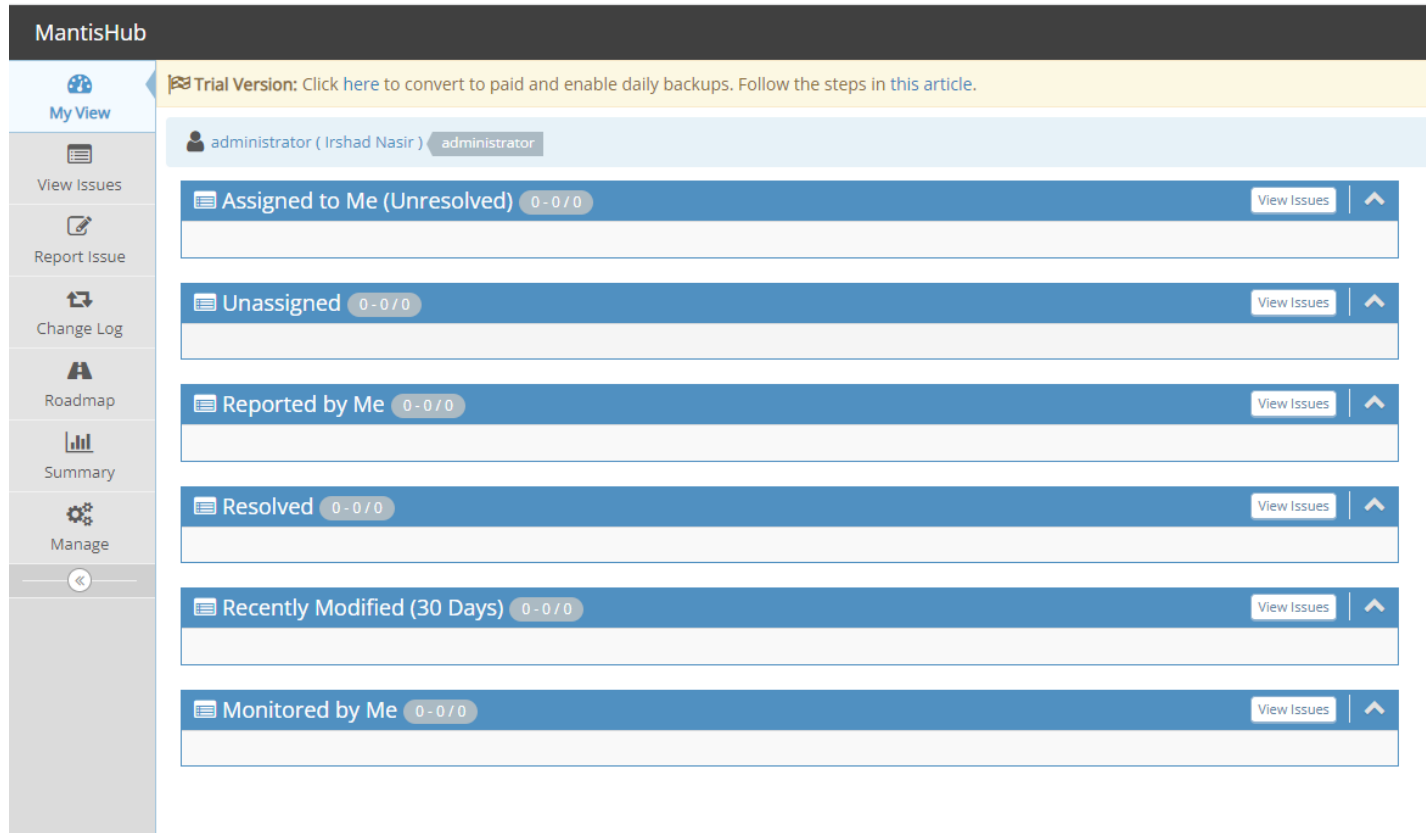
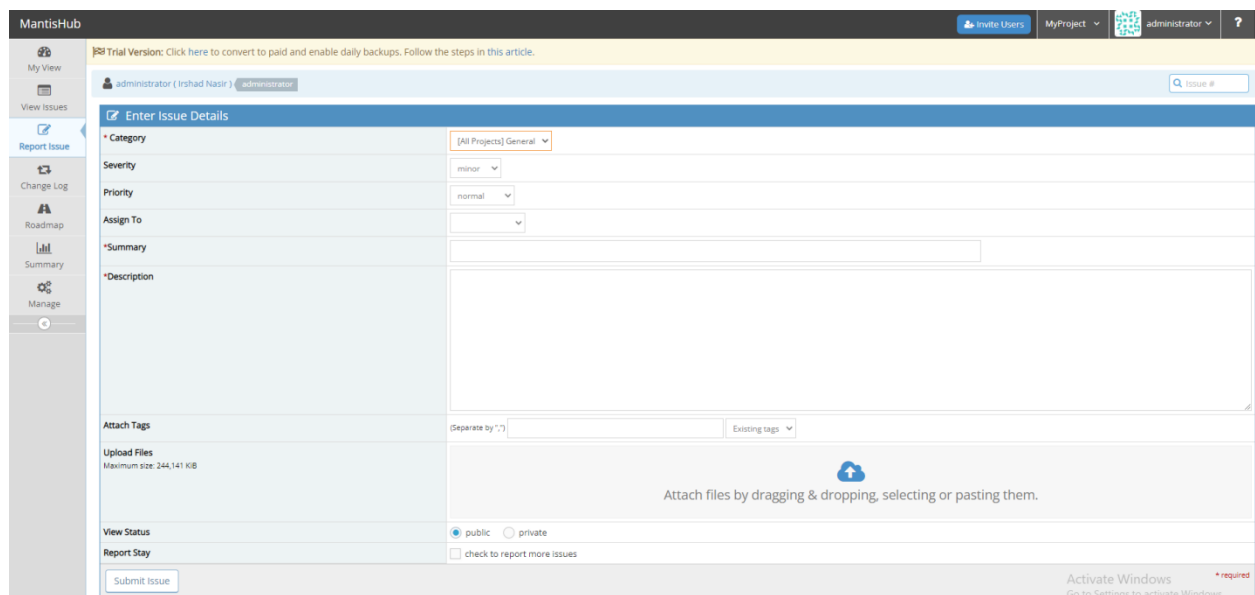


Figure 2-Interface for report an issue:



Example-II: Logging Defect in Mantis

- Provided Link:

https://www.testqse.mantishub.io/login_page.php

Figure 3- Interface for Adding Users

The screenshot shows the 'Create New Account' form in the MantisHub interface. The form includes fields for Username, Real Name, E-mail, Access Level (set to 'reporter'), Enabled (checked), and Protected (unchecked). A 'Create User' button is at the bottom. The top navigation bar shows 'Report Issue', 'Invite Users', 'MyProject', and 'administrator'. The left sidebar has icons for 'My View', 'View Issues', 'Report Issue', 'Change Log', 'Roadmap', 'Summary', and 'Manage'.

Figure 4-Assigning issues to users.

The screenshot shows the 'View Issue Details' page for issue ID 0000002. The page includes a table with issue details and a summary section. The table has columns for ID, Project, Category, View Status, Date Submitted, and Last Update. The summary section includes fields for Reporter, Priority, Status, Assigned To, Severity, Resolution, and Attach Tags. The bottom of the page has buttons for 'Edit', 'Assign To', 'Change Status To', 'Monitor', 'Stick', 'Clone', 'Close', 'Move', and 'Delete'.

ID	Project	Category	View Status	Date Submitted	Last Update
0000002	QSE	[All Projects] General	public	2017-05-12 15:02	2017-05-12 15:02

Reporter	administrator	Assigned To	administrator
Priority	immediate	Severity	block
Status	assigned	Resolution	open

Summary 0000002: Login not working

Description Click on login button->valid credentials not working

Tags No tags attached.

Attach Tags (Separate by ",") Existing tags

Mantis

Mantis is one such tool used for keeping logs and tacking details of the defects. This tool is also available on the cloud and it makes collaboration among team members and other stakeholders easier and faster. So, mantis bug tracker is a web-based bug tracking tool which is free and open source. It is most commonly used tool to track software problems.

Updating the existing defect

First of all, login to mantis as an administrator. Admin can add resources and team members in mantis. So, there will be multiple options here and you can change the existing defect as well as can also change the existing user. As an administrator, you can view the issues. For this purpose, click on the View Issues button and you can see the already logged issues. To view the issue, click on the particular ID of that issue. So, here you can see the detail of that issue and can assign it to the developer to fix it. By clicking the Edit button, you can update the description of defect like it is not working with the valid credentials or etc and assign it to the developer. At the end you will click on the Update User button. Now, it will assign the issue to the developer.

Implementing defect lifecycle

A defect lifecycle is made up of a series of specific stages – commonly referred to as workflows – which a defect must go through. The defect lifecycle begins with the discovery of the flaw and concludes with the closure of the defect and the assurance that it will not be repeated. While different software development firms may take different processes, defect management technologies come with pre-defined steps for managing the defect lifecycle.

What is meant by software testing?

Software testing is the process of running a software product with the goal of ensuring that it produces the anticipated results and meets all of the criteria and business objectives. Another goal of software testing is to identify problems and bugs so that they may be documented, evaluated, and fixed.

What is meant by software testing in a layman language?

In a layman language, software testing is all about comparing the product's actual and expected behaviour. Software testing can provide a concrete and objective view of the quality of the product.

CS611 Handouts

Topic 174-230

Need of testing

In a broader sense, testing is one of the most significant aspects of software quality assurance operations since it ensures that the system is working as planned. Testing offers objective information about the quality of a software product, allowing stakeholders to have confidence in it. It also discloses any dangers associated with the product's execution, if any exist.

Natural way of testing

The most natural method is to dry-run the software in a controlled or simulated environment to eliminate software defects/bugs prior to product deployment.

Artifact

Whenever we say an artifact, it means a document or some visualization. In other words, we can say that something noticed in an experiment or investigation which is not normally exists.

Software Artifact

Everything, which is involved in the development of a program, is referred to as a software artifact. Mock-ups, design documents, test matrices and other items developed during the development process are displayed.

Artifact Level Testing

The software program or code written in various programming languages is the primary software artifact to be tested. Writing and executing test cases built from use cases is the most common method of testing. For a specific test, a test case is a document that outlines the testing activities, pre and post conditions, expected system behaviour, and outcome. While use cases describe all the steps that final user will be performing to complete the task and those steps become the input of the test cases which elaborate those steps and actual & expected results against each one.

Test Planning and Preparation

In this activity, the objective is to set the goals for testing, select the best and appropriate testing strategy/methodology, and prepare specific test cases and the general test procedure.

Test Execution

In this step, we execute and implement the testing strategy and measure the behavior of the product.

Analysis and Follow-up

This step includes analysis of results to determine if a failure has been observed, and if so, corrective actions are taken and monitored to ensure the removal and reproduction of defects.

Functional Testing

Functional testing is a type of software testing, which is more concerned about verifying that the software and all of its functions are performed in total conformance with the software requirements.

Focus of functional testing

Its focus is more on the external behavior of the software i.e. to test the functionality and the features of the product that how functionality is achieved i.e. programming or coding is not a matter of question in functional testing.

Functional testing as abstract level testing

It is usually called as abstract level testing because it ensures that software is giving the desired output to the end user.

Structural Testing

Structural testing is the total opposite of the functional testing and it is more focused on the internal implementation details of the software product. The purpose of this type is to test the different programming structures and data structures used in the program. Structural testing verifies the internal units of the software product like programming/code structure, blocks, data structure and all relevant interconnections are implemented correctly.

Structural testing as concrete testing

Structural testing is called as concrete testing from the code view point because the code is tested at statement level or module level or sub-system level.

Black Box Testing

Functional testing and black box testing are two terms used interchangeably. Functional testing is usually achieved by performing black box testing. In the simplest form, black box testing is an ad-hoc running and execution of the software system to observe any difference between expected and actual performance and in case of gap or variance from the desired results this process is repeated to eliminate the possibility of hardware failure causing the difference.

Specification Checklist

Specification checklist is a good way to perform this testing. Specification checklist describes the functionality and features of the product with sample input and expected output data. Smoke Tests are the most abstract level of testing that aims at ensuring that the most important functions work.

How can we do black box testing?

As per the interface (a simple calculator) shown in the video, this software takes some input and gives some output and what's going on inside is unknown. Some of the test ideas would be something like:

- To explore supported values, perform test for floating point number, integers, and boundaries of integers.
- In case of division, Divide by 0 would be a classic error.
- Use mouse, keyboard, type and paste to explore the possibility of entering values.

How can we achieve black box testing?

As per the example in video, imagine we are testing a Date Class with a DaysInMonth (month, Year) method. What are some condition and boundary test for this method?

Possible answers include:

- Check for leap year (every 4th year)
- Try years such as: even 100s, 101s, 4s, 5s
- Try months such as: June, July, Feb, Invalid Values
- **White Box Testing**
- White box testing is performed by the developer himself as it required extensive knowledge of code and flow of code. White box testing covers branch level testing (if-else, loops, switch) and Path level testing (all possible paths). Structural testing is achieved by performing white box testing and its aim is to verify the correct internal implementations i.e. programming/code, data structures, blocks, relationships and interconnections are implemented properly. This test is done by executing and observing the program/software behavior related to these specific units. Different debugging tools are used to test coverage of the specific statement level code. This means that tester is able

to see if all or a specific statement has been executed or not, and if yes, is it working properly and giving the desired output or not. In this way the error is identified by its location.

- **White Box Testing (Statement Level and Branch Level Testing)**
- Statement Level and Branch Level testing ensure coverage to statements and branches (if-else, while, for, switch).
- Float calculate (int a, int b, int c)
- {
- Float e=0.0
- if
- (a==0) return
- e; int x=0;
- If (a>b && c<b&&c>a) -- a=4, b=3, c=2
- {
- X=a;
- }
- E=1/x; return e;
- }
- **White Box Testing (Path Coverage)**
- Path Coverage: All the possible paths (decision points to be tested). Path Level Testing ensures maximum coverage of code.
- Int main ()
- {
- Int
- a,b,c; C=
- a+c;
- If (c>100)
- { Cout <<"It's done"
- ;} If (a>50)
- {Cout<<"It's Pending" ;}
- }

- **Reviews in Testing**

- The review is another kind of software testing, which is performed before executing the test cases or executing the software code to see if it's working properly. This is a static testing method in which the software artifact like code is manually examined without execution with the goal of early defect detection and removal. In this way, the mistakes overlooked in the early stages of software development are identified and corrected which ultimately improve the software quality.

Rationale for Review

- Reviews in the context of testing are a time saving activity and need no formal planning or resource utilization because it is more of an informal testing. Defects identified in at this stage are easy and less costly to handle and rectified as compared to defects identified in formal types of testing. Reviews include code and design walkthrough and it is usually done by more experienced technical people who are not the author of the code. It can take shape of informal code walkthrough that increase the knowledge sharing as well.
- **Inspection**
- It is a more formal type of static testing (Reviews). In inspection, the review activity is normally led by a trained facilitator or moderator who is not the author of the code. This formal activity aims at evaluating the level of compliance with specific rules, principles or the best practices of software development. All inspection activities are well documented and all the defects identified in this stage are logged in defect logging sheet.

Example of Inspection

Inspection for code is basically removal of redundant or unwanted code at max. Following are some of the questions asked during inspection activity.

-
- Is the compilation list free of warning messages?
- Are there any uncalled or unneeded procedures?
- Can any piece of code be replaced by calls to external reusable components or library functions?
- Will requirements at execution time be met?
- Is the code well-structured, consistent in style, and consistently indented?
- Are there any blocks of repeated code that could be condensed into a single procedure?

Are symbolic used rather than "magic number" constants or string constants?

Test Check List

- Test Checklists are steps to be executed every time testing is to be performed. It consists of standard steps to list down all activities and tasks to be performed before the test and during the test. This is not a kind of functional testing which is totally different. This is simply a list of activities to be performed to ensure the readiness for formal QA tests.

Example-I: Test Check List (Web testing checklist for data security)

Below is a sample web testing checklist for data security. These are only sample questions and may include other as well.

-
- Are data inputs adequately filtered?
- Are data access privileges identified? (e.g., read, write, update and query)
- Are data access privileges enforced?
- Have data backup and restore processes been defined?
- Have data backup and restore processes been tested?
- Have sensitive and critical data been allocated to secure locations?
- Have a data archival and retrieval procedures been defined?

Example-II: Test Check List (Web testing checklist for performance)

Below is a sample web testing checklist for performance. This is to be noted that check list does not include functional testing. Testing Checklist is very critical to test the overall behavior of the System.

-
- Has the database capacity been identified?
- Have anticipated growth data being obtained?
- Is the database self-contained?
- Is the system architecture defined?
 - Tiers
 - Servers
 - Network
- Have the various environments been created?
- Is load balancing available?

What is configuration management?

- In software engineering, configuration management is the collection of the best practices of tracking and controlling changes in the software development. Multiple teams and resources work on different parts of the software at the same time and the work need constant changes. So the configuration management team helps them in keeping record of the different versions of their work and determining what was changed and who changed it and make them able to revisit the changes if needed. Configuration management is closely associated with QA as a QA team receives shipment of software code from the configuration management team and vice versa.

Why configuration management?

Multiple developers and programmers work on different parts of the software on a shared code base towards single working software, different stages like development, testing and production required different version of the same software and SCM make sure that a specific version is available with all of its change history. CM is intended to eliminate the confusion and error brought about by the existence of different versions of artifacts. There are many software products available in the market like VSS, TFS and Git that are used to manage changes and different version of the code.

Example of configuration management

The need is to have a single repository to manage all the releases of the software and CM is tasked with fulfilling of this need. All the release activities are coordinated by an SCM team and then the code base is handed over to QA and then to the customer is passed. But the customer may not like changes made in a certain release, as in release 2 in the image on the right (as per the image in video). Thus, versioning and keeping backups of all the releases makes it easy to revert back any changes. So SCM also manages what changes are requested by which customer.

Product and configuration management

A single software product may have different version based on the clients need and all the versions may have multiple code bases. Change requests made by one client is not visible to other clients and those changes must not impact other clients as well. So it is the task of SCM to manage customer requests and determine what version of software is being used by that client. Without Configuration Management it is impossible to track multiple software versions and their multiple code bases. Changes in software products are facts either for error removal or just for product refinement and it is the responsibility of SCM to manage all changes in complete details.

Test Planning and Preparation

This is one of the most important activities in the whole testing process because most of the decision related to testing is made at this stage which determine the quality of the software testing. The decisions made in at this stage are based on several key questions and answers to those questions decide the testing strategy. Few of those questions are as follows:

-
- What are the objectives and overall goals of the testing activities?
- What specific areas/features of software are to be tested?
- What would be the exit criteria or when to stop testing?
- In what environment(s) the software is to be tested?
- There are many other questions as well that set the direction of software testing

Test Cases Basics

Test Cases vs. Functional Requirements

- Every software product has some functional requirements stating the nature of function it will perform or what the system is supposed to accomplish. Test case defined all the steps to test the functionality of the product. In case of closure of the test, the product is considered to be providing all functionalities as per the requirement. Test cases can refer to multiple functionality depending on use cases.

Example-I: Test Case Template

		Project Name		XYZ			
		Module Name (In case of Product)		NA			
Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass /Fail	Comments
TU001	Check Customer Login with valid Data	i. Go to site http://www.test.com ii. Enter User Id iii. Enter Password iv. Click Submit	User id = guru99 Password = pass99	User should Login into application	As Expected	Pass	
<i>Note: There can be multiple tests against one scenario.</i>							

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail	Comments
TU02	Check Forget Password	i. Go to www.test.com ii. Click on Login Page iii. Click on Forget Password Link iv. Email Text box to PopUp v. Enter Valid Email vi. Reset Password URL to be email vii. User Click on Reset Password URL viii. User Should set password and conform new Password ix. Customer should be able to login with new Password	Email for reset Password sherazpervaiz@gmail.com	User should be able to login with new password	Failed	Failed	Step - ix Fail
Note: If individual Step Fail then overall test case Fails.							

Example-II: Test Case Template

Project Name	XYZ
Module Name (In case of Product)	NA

Example-II: Test Case Template

		Project Name		XYZ			
		Module Name (In case of Product)		NA			
Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail	Comments
TU02	Check Forget Password	i. Go to www.test.com ii. Click on Login Page iii. Click on Forget Password Link iv. Email Text box to PopUp v. Enter Valid Email vi. Reset Password URL to be email vii. User Click on Reset Password URL viii. User Should set password and conform new Password ix. Customer should be able to login with new Password	Email for reset Password sherazpervaiz@gmail.com	User should be able to login with new password	Failed	Failed	Step - ix Fail
Note: If individual Step Fail then overall test case Fails.							

Product Testing

A product is a software application that is developed by a company with its own budget & requirements from market standards. Product testing is for the organization's own product and based on the organization's own requirements and market standards. Product testing is on-going activity. Test cases of specific products are regularly updated as per release.

Project Testing

A project is a software application that is developed by a company with the budget & requirements from a customer. Project testing is for client and based on requirements from the client. Primarily project testing is a one-time activity and ends when the product is delivered to the client. Test cases in projects are usually specific under specific environment.

Unit Testing

The unit is a small and testable part of software like functions, classes, or interfaces. Usually it has one or more input and single output. Unit testing is a level of software testing where individual units of software are tested and algorithms are verified independently. The purpose is to validate that each unit of the software is performing as per requirements and design. This is considered to be the very first level of testing and more of a development process as it is usually performed by developers itself. As units are tested individually and independently, unit test provides a cheap way to verify the functionality without impacting other functionality.

Example of Unit Testing

Unit Tests ensure stability and functionality of software from beginning, even before the start of formal testing.

```
public class Calculator {  
  
    public int add (int x, int y)  
  
    {  
  
        return x+y;  
  
    }  
  
    public int sub (int x, int y)  
  
    {
```

```

return x-y;

}

}

public class test

{

Public static void main(String [] args)

{

Int result=0;

Calculator A = new A();

Result=a.add(5,9); System.out.println(result);

Result=a.sub(5,9); System.out.println(result);

}

```

Impact of Unit Testing

Unit testing ensures early defect removal as it is very easy to look for defects individually and independently in all units of software code. As it is done in isolation, it becomes easy to debug the code to prevent defect reproduction and the unit test efforts are done at the developer's level who can easily understand the code. Further, the cost of fixing a defect detected during unit testing is lesser in comparison to that defects detected at higher levels.

Sanity (Integration) Testing

The next level of testing is sanity in which multiple units are tested together. As mention in the configuration management section, multiple developers work on different modules of the same software and then SCM merge the code. That merged code is then handed over to developers again to perform sanity testing. The bottom line is, if the units are working individually and independently, they should be working in combination too and should be performing as expected. The focus of sanity testing is on integration of different components to work together so components with their interconnections are examined and evaluated.

Example of Sanity Testing

As you see the example of a product database in video, where a user can login to add products and product categories and can search the previous added products as well. There were two units here as you see in the video. You also see the situation of these two units before and after integration testing. So, these two units are working independently and they must work when integrated with each other.

Sanity Testing Reporting

Sanity test reporting provides overall status of the integration testing and it helps in identifying critical areas that are to be fixed. Features are identified with data that require bug fixing. The bugs are identified as critical, major, medium and cosmetic for all the units/features.

Example of Sanity Testing Reporting

As per the image explained in the video, let's imagine a product with 4 features namely Registration, Booking, Payment and Reports. Sanity or integration testing performed on the mentioned features might look like the image below in which bugs/defects are identified as per their severity level.

	Registration	Booking	Payment	Reports	Total
Critical	6	7	5	7	25
Major	4	5	2	4	15
Medium	6	8	2	4	20
Cosmetic	1	2	1	1	5
Total-->	17	22	10	16	65

Reflect integrated status of multiple features

Feature Level Testing

Functions of any software product are functional requirements that are developed as a unit and depending on feature size and nature there can be multiple underlying units. Feature level testing is aimed at making sure that the software and its features work properly to meet all the intended specifications and requirements. In simple words, it's a test for software

functionality where each function, that might be consisted on multiple units, is evaluated to see if it is working as required and designed.

Example-I: Feature Level Testing

Imagine a product with a feature that enables a user to book a hotel room via a website. That feature will be consisted of the following units.

- - **Feature Name: Booking of Hotel Room via Website**
 - Registration of User
 - Searching of Rooms in Hotel
 - Selecting a particular Room in a Hotel
 - Filling – in payment details
 - Verification of Payment Details
 - Generation of Unique Booking Id
 - Sending Booking ID to User via Email

Each feature can be developed as one or multiple units and in feature level testing all features of the product are to be tested. Feature level testing requires integration testing to test feature, if there are multiple units.

Example-II: Feature Level Testing

Consider another product with hierarchy management feature. The units for this feature will be as follows and they all will be tested in feature level testing.

- - **Feature Name: Hierarchy Management**
 - Register Hierarchy
 - Add Hierarchy Name
 - Add Hierarchy Level
 - Add Root of Hierarchy
 - Root Hierarchy can have a child

System Level Testing

- This is an advanced level of testing and the fully integrated final product is tested and evaluated in the exact environment in which it is to be used by the end users. The overall architecture of the product is evaluated along with its ability to satisfy the business requirements. Software evaluation is done on the basis of the user's point of view and has very less to do with code design and structure. System testing is performed by independent testers who do not have a

ny contribution in development activities. System Testing is very important because it verifies that the application meets the technical, functional, and business requirements that were set by the customer.

- Child can be root node also

Stress Testing

It is another form of system level testing and the purpose of this test is to evaluate the effectiveness and stability of the software under unfavorable conditions and that mostly includes quantitative test to gather stats related to frequency of crash and errors. During the test, adverse and unfavorable conditions are deliberately created and maintained and fully integrated system is evaluated throughout those environments to measure their breaking point. A fine example is clicking a website with multiple concurrent to test the load.

Regression Testing

This test is performed to verify that with the addition of new feature the previously developed and tested system still perform correctly. The complete software development project includes maintenance, defect fixing, optimizations, error correction and that result in continuous update in the product. The goal is to make sure that with the addition or change in one part does not impact other parts and the newly added features are fully integrated with the already developed system. The regression testing verifies that the goal is achieved or not.

Example of Regression Testing

Lets consider development of a product and the following features were developed with the release 1.0 and 1.1.

Feature No.	Release 1.0	Release 1.1
1	Login	Login
2	Product Listing	Product Listing
3	Shopping Cart	Shopping Cart
4	Payment via Credit Card Only	Payment with Discount Card using PayPal also
5	---	Discount Card

In release 1.1 two features were added or updated (discount card and payment option with discount card or PayPal) and that should not impact the already developed features.

Test Reporting

Test reporting is the most important artifact of the whole software testing activities as it shows the overall status of all the testing activities performed throughout all stages. There are different levels of tests at different stages and it becomes extremely important to track all tests performed and their results. Many different shapes and formats are available for overall status of test activities and one such format is Test Dashboard.

Test Dashboard

Test dashboards are statistical analysis of overall testing activity in progress. The data is statistically and visually presented in this dashboard and they help testers a lot in making useful decision regarding testing strategy. By using the Test dashboard, the QA team can monitor test activities, report on progress, find gaps in test coverage, and identify test areas that may require additional investigation.

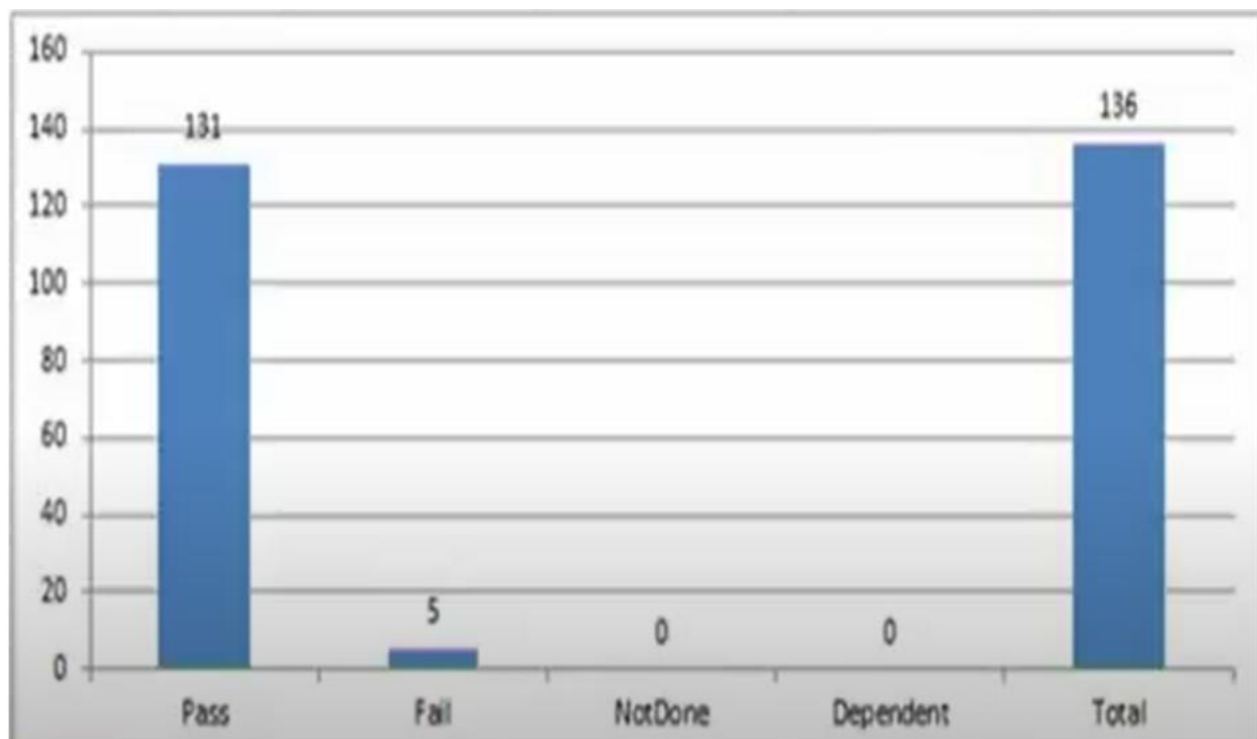
Example-I: Test Dashboard

The test dashboards are able to tell the count of defects of all statuses (closed, fixed, hold, NotAFix etc.) separately for all the features of the product, as shown in the image below.

Overall Status of 4.6.4									
Bug Status Report									
Features	Closed	Fixed	Hold	NeedMoreInforma	Not A F	Not Reproduc	Open	Verified	Grand Total
FrameWork		1	2				3		6
BackEnd	13	1	1	2	3	13	45	1	79
Search Engine							1		1
Execution Engine							1		1
Layering							1		1
Help							1		1
Layering and Decluttering							1		1
Grand Total	13	2	3	2	3	13	53	1	90

Example-II: Test Dashboard

The data in the test dashboards can be graphically presented to make it easy to understand. The example is shown below.



Release Notes Basics

Release notes are documents that are distributed with the software release revealing what is included in the software i.e. the features and steps on how to install and deploy the product. The list of known defects that are not fixed is a part of the release notes while mentioning what defects, deficiencies are fixed in a particular release. Another important thing that this report contains test results and information about the test procedure to increase client or end user confidence in the product.

Example-I: Release Notes

As you see, template of release notes has already been explained in the video. This activity is performed after the closure of project testing cycle. Release notes include outstanding issue in the product. It also includes the limitations of the product.

Example-II: Release Notes

As you see, another aspect of release notes has already been explained in the video. In this activity, the road map of testing is also shared with the client.

Post Production Issues

Post production issues are reported to the end-users once when staging environment is productionize. Productionalizing an environment mean to deploy staging server code base to Production Server which is provided to the client or the client has some sort of arrangement with the Vendor for deployment. Usually good QA will ensure that there will no critical bug on Production side but even good QA will not ensure 100% bug free code that's just possible. Production issues are those issues which are reported to end-users can be of low priority from development but from end-user viewpoint they might take high-priority which are needed to be addressed.

Types of Post Production Issues

The way to handle pre-production issues and post-production issues is simply different due to the reason that end-user is usually not involved during development. During development in-scope bugs are fixed and shipped to ensure max compliance with agreed requirements. In Production it is possible that End-user or client may report issue which is not a bug, but needed to be fixed, i.e. WishList or issue is there due to latest build i.e. Ripple Effect. For WishList vendors usually do analysis, estimates and share timelines but these are paid changes but for Ripple Effect cost is bear by Vendor because somehow bug gone undetected during QA. First step to proceed when issue is reported from Production is to classify it either a Bug or Wish List and then accordingly steps are taken.

Hot Fixes

Hot fixes are issues reported from Production (Live Site) which can either be WishList or Bug but in either case we have little time to respond for Hot Fix due to importance and criticality. There are scenarios where end-user or client is not worried about the cost factor and just expect to fix the issue as soon as possible as it may be impacting business. In this scenario usually complete QA process is not executed majorly due to the time factor and the team is dependent on unit testing and Code Reviews to fix hot fixes rather than regression testing.

Patch

Patch as opposite to Hot Fix are those issues which are critical, but the client can wait for a short period of time to get them fixed. The difference between Patch and Release is that,

the time frame of Patch to deliver is usually less than Release but more than Patch and development team to do the proper estimation which includes QA cycles also and then share deadline. Usually leftover bugs or WishList or both are delivered as a patch after Major Release.

Example of Patch

Assuming there were requirements from the client to develop a website where user registration will be performed by clicking on the registration page and then the user will fill-out the registration form and submit the details and then the user will be able to login with credentials. After Project is delivered to the client, there is need to integrate registration using Social Media, patch is to identify this request either as WishList or bug, in this situation software requirements and specifications document or SRS because to negotiate with client verbal communication are usually enough to convince client in case of conflict. In this scenario, if client accept it as a WishList then estimates and deadlines will be shared accordingly.

THE END